

Einsendeaufgabe UML-E1: Eigenes UML-Werkzeug wählen und Diagramme erstellen

Für die Auswahl von UML Tools habe ich die KI Chat GPT befragt mit folgendem Prompt:

“Bei einem für mich passenden UML Werkzeug würde ich gern etwas weitersuchen. Welche gibt es da? Nenne mir 20. Bitte nur kostenlose oder für Studenten kostenlose.”

Die Antwort lautete:

🔑 Klassische GUI-Tools (Drag & Drop)

1. diagrams.net (draw.io)
2. Lucidchart (Free Version)
3. Visual Paradigm Online (Free Plan)
4. Visual Paradigm Community Edition (Studenten kostenlos)
5. yEd Graph Editor
6. Creately (Free Plan eingeschränkt)
7. GenMyModel (Free Plan)
8. UMLet
9. UMLetino (Web-Version von UMLet)
10. StarUML (Testversion / teilweise frei)

🔑 Vorteil: einfach zu bedienen, ideal für deine ESA

🔑 Textbasierte UML Tools (sehr gut für Teil B)

11. PlantUML
12. Mermaid
13. Graphviz
14. Structurizr DSL

15. Kroki (Renderer für viele Diagramm-Sprachen)

👉 Vorteil:

- perfekt für GitHub / AI / Automation
 - z. B. Mermaid läuft direkt in Markdown
-

👉 Open Source / Entwickler-orientiert

- 16. JetUML (leichtgewichtig, Open Source)
 - 17. BOUML (inkl. Code-Generierung)
 - 18. Modelio (Open Source UML/BPMN Tool)
 - 19. Papyrus (Eclipse UML Tool)
 - 20. ArgoUML
-

👉 Wichtige Erkenntnis (für deine ESA!)

👉 Es gibt 3 Kategorien:

- GUI Tools → einfach (draw.io)
- Text Tools → modern/dev (PlantUML, Mermaid)
- Profi Tools → komplex (Papyrus, Modelio)

Teil A

Ich bleibe thematisch bei meinem Projekt der Migration der Auftragssteuerung von SAP R/3 nach ServiceNow.

Für Teil A habe ich das Tool diagrams.net gewählt, da es kostenlos nutzbar ist, UML-Diagramme unterstützt und sich einfach im Browser bedienen lässt.

Ich habe mich für folgende Diagramme entschieden:

1. Klassendiagramm
2. Anwendungsfalldiagramm
3. Aktivitätsdiagramm
4. Sequenzdiagramm
5. Zustandsdiagramm
6. Komponentendiagramm
7. Deploymentdiagramm

Diese Auswahl ist passend, weil sie verschiedene Sichtweisen auf das System zeigt: Struktur, Nutzerrollen, Abläufe, Kommunikation und Statuswechsel.

Ausführlich habe ich gleich das erste, also das Klassendiagramm dargestellt, da es das gängigste Diagramm in diesem Zusammenhang darstellt.

Klassendiagramm mit folgenden Klassen:

1. Auftrag
2. Auftragsposition
3. Benutzer
4. Bearbeiter
5. Fachbereich
6. Status
7. Rückmeldung
8. Kommentar
9. SystemSchnittstelle

Beziehungen

Auftrag enthält 1 bis viele Auftragspositionen
Auftragsposition gehört genau zu einem Auftrag
Auftragsposition wird einem Bearbeiter zugewiesen
Bearbeiter gehört zu einem Fachbereich
Auftragsposition hat genau einen Status
Auftragsposition kann 0 bis viele Rückmeldungen haben
Auftragsposition kann 0 bis viele Kommentare haben
Systemschnittstelle überträgt Auftragsdaten zwischen SAP R/3 und ServiceNow

Komposition / Aggregation

Komposition:
Auftrag besteht aus Auftragspositionen

Aggregation:
Fachbereich umfasst Bearbeiter

Attribute Beispiel

Auftrag:
auftragsnummer, kunde, erstellDatum

Auftragsposition:
positionsnummer, beschreibung, prioritaet

Benutzer:
benutzerId, name, rolle

Bearbeiter:
mitarbeiterNummer, zuständigkeit

Olaf Gustke
BHT S26 , Wirtschaftsinformatik Online B.Sc.
UML – E1

Fachbereich:
bereichsName, kuerzel

Status:
statusName, farbwert

Rückmeldung:
stunden, rueckmeldeDatum, bemerkung

Kommentar:
text, erstelltAm

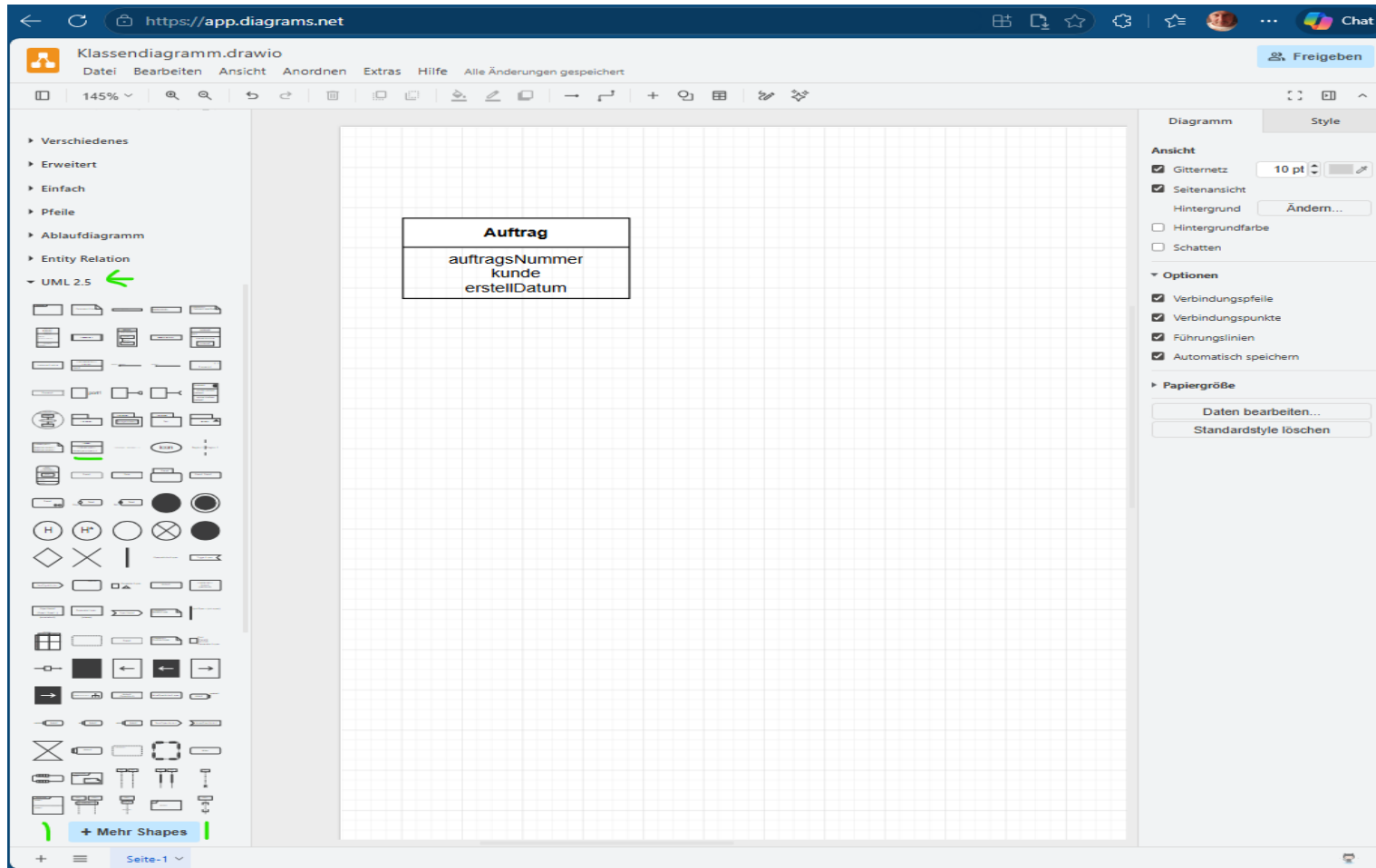
SystemSchnittstelle:
quellsystem, zielsystem, schnittstellentyp

Die größten Probleme und Herausforderungen bei der Erstellung der Diagramme hatte ich bei dabei überhaupt erstmal die verschiedenen Diagrammtypen auseinanderzuhalten und deren korrekten Notationen einzuhalten. Dann auch ganz banale Problem, wie das Felder sich einfach schwer im Format so anpassen liessen, wie es ideal ist. Des weiteren, dass Linien auch mal nicht ganz gerade ansetzbar waren am Anfang und ich damit dann nicht ganz zufrieden war.

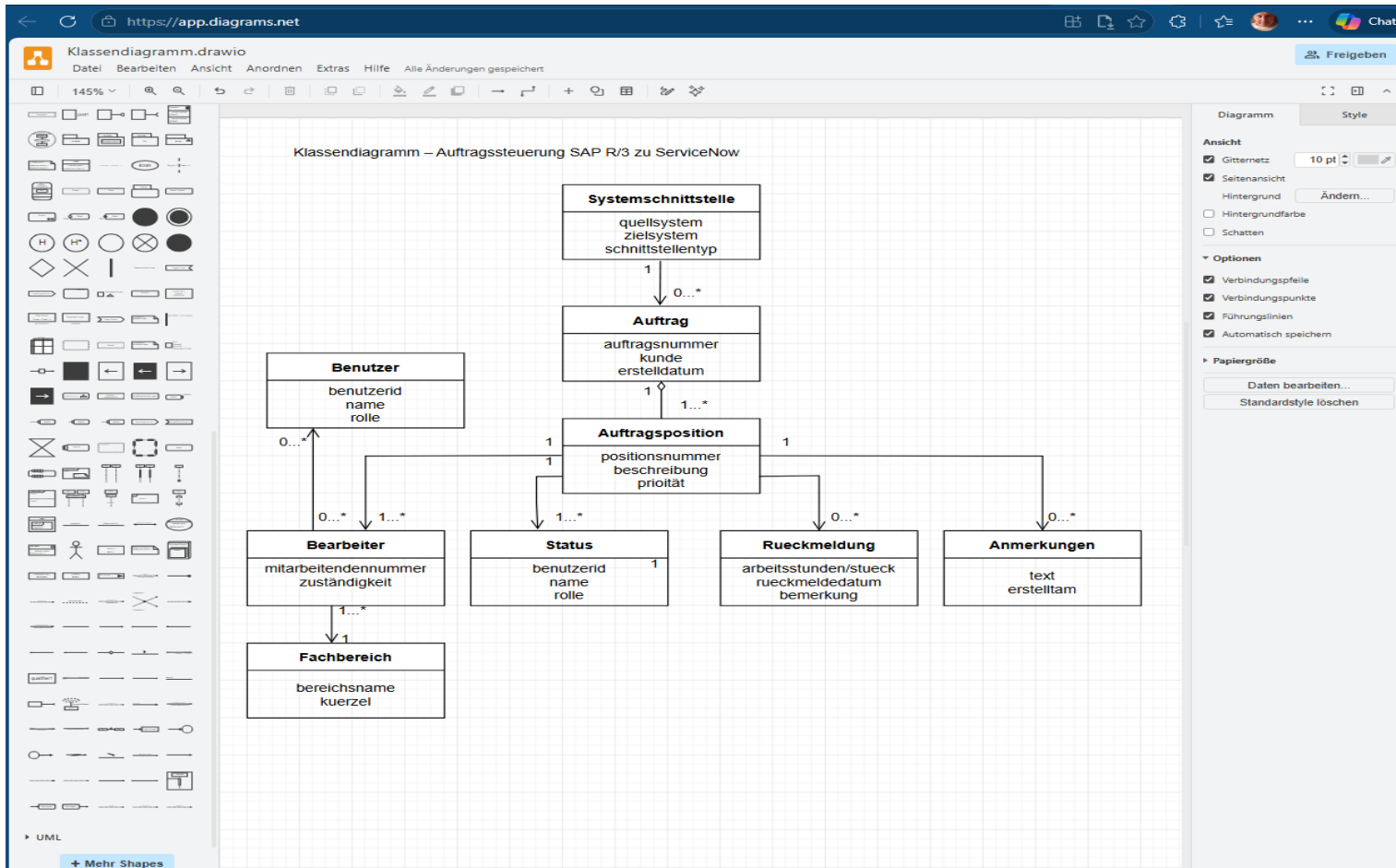
Auch die Beschriftungen waren eine Herausforderung, da der Text nicht immer genau dahin wollte, wie er sollte. Und auch bei der Pfeilen, gibt es in draw.io wie es früher hier mehr Möglichkeiten einfache Pfeile oder Verbindungen wie Linien oder auch Connectoren zu nutzen.

Im Laufe der vergehenden Zeit wurde es aber besser und ich kam besser rein. Zur Hilfe und Erklärung mancher Funktionen habe ich natürlich KI auch genutzt.

Als nächstes habe ich diagrams.net geöffnet, was mir schon als draw.io bekannt war. Hier rechts unten unter mehr shapes ausgewählt UML 2.5 als die modernere Variante von UML. Hier zum Beispiel dann das Symbol "class" gewählt und rechts auf den Bildschirm gezogen und dort ausgefüllt.

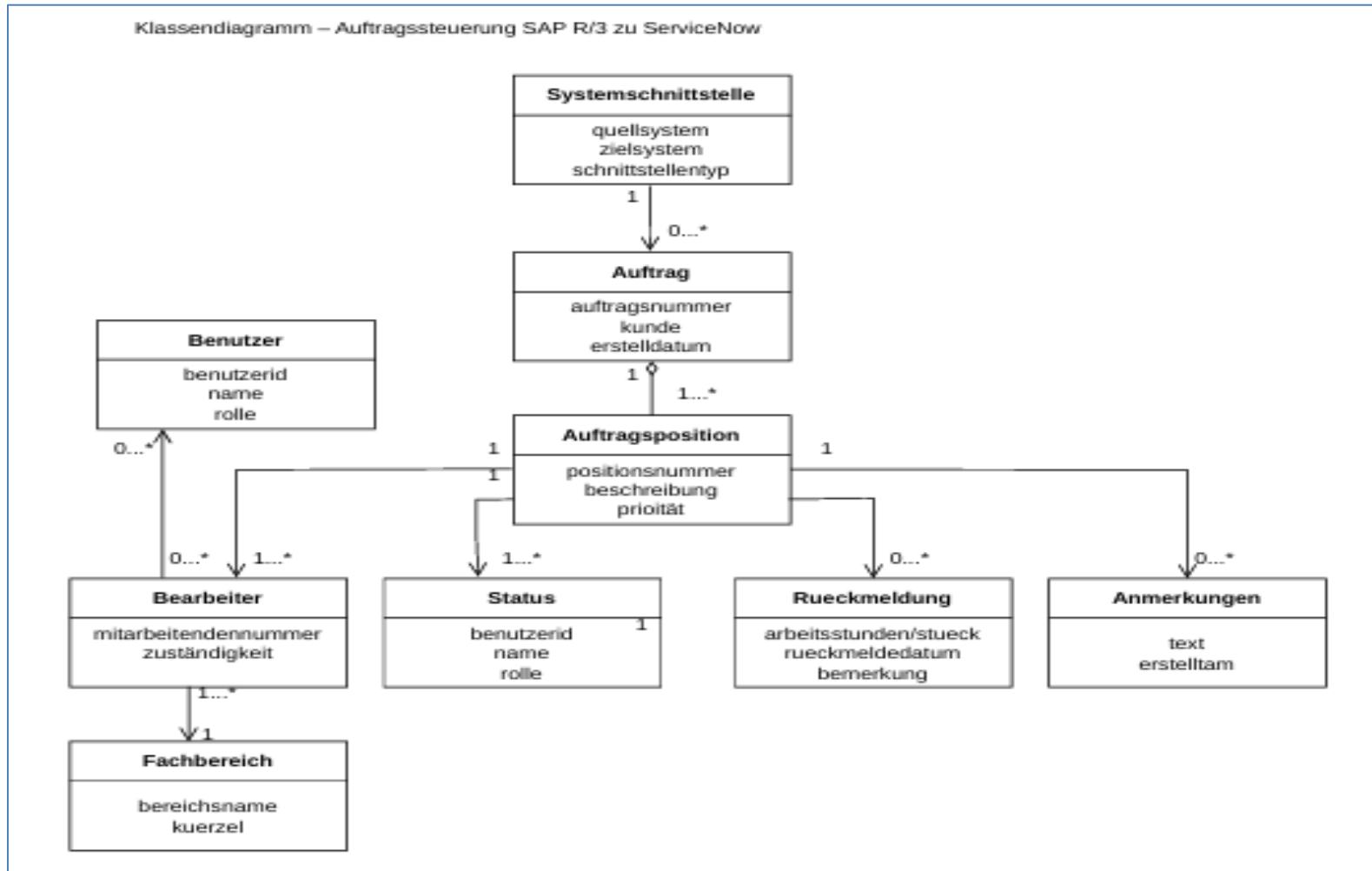


So sieht mein fertig erstelltes **Klassendiagramm** mit <https://app.diagrams.net> aus:



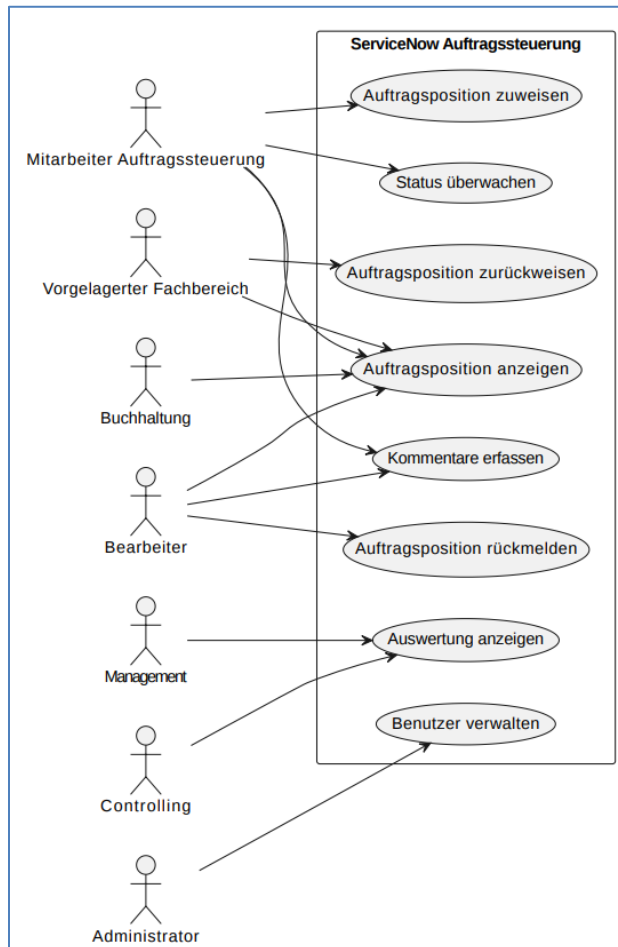
Das Klassendiagramm zeigt die wichtigsten fachlichen Klassen der Auftragssteuerung sowie deren Beziehungen untereinander. Dieser Diagrammtyp eignet sich besonders gut zur Darstellung von Datenstrukturen, Kardinalitäten sowie Aggregationen und Kompositionen innerhalb eines Systems.

Exportiert als PDF und per Screenshot kopiert sieht es zum Beispiel wie folgt aus:



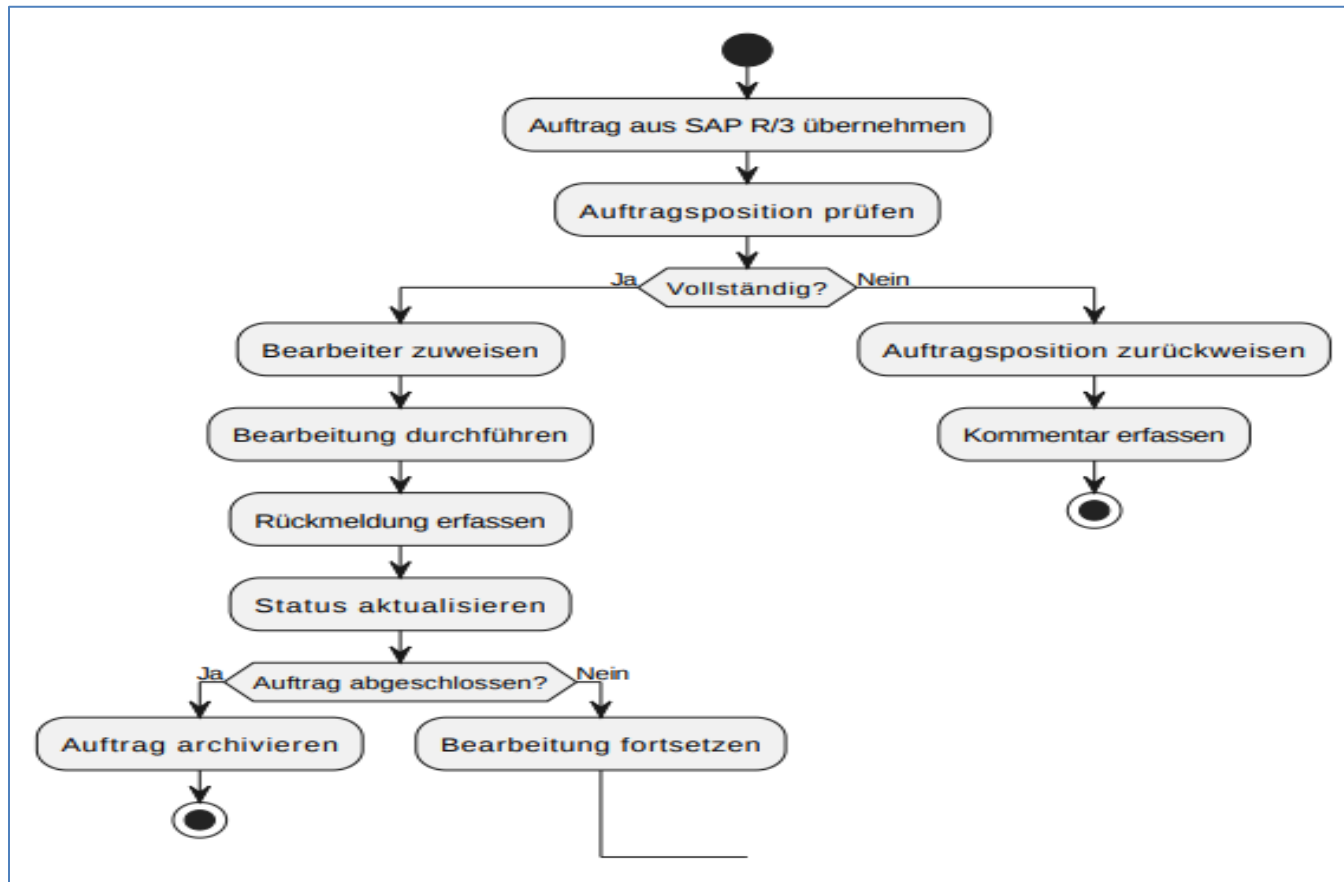
Anwendungsfalldiagramm

Das Anwendungsfalldiagramm zeigt die beteiligten Akteure und deren Interaktion mit der ServiceNow-Auftragssteuerung. Dieser Diagrammtyp eignet sich besonders zur Darstellung fachlicher Funktionen und Benutzerrollen innerhalb eines Systems.



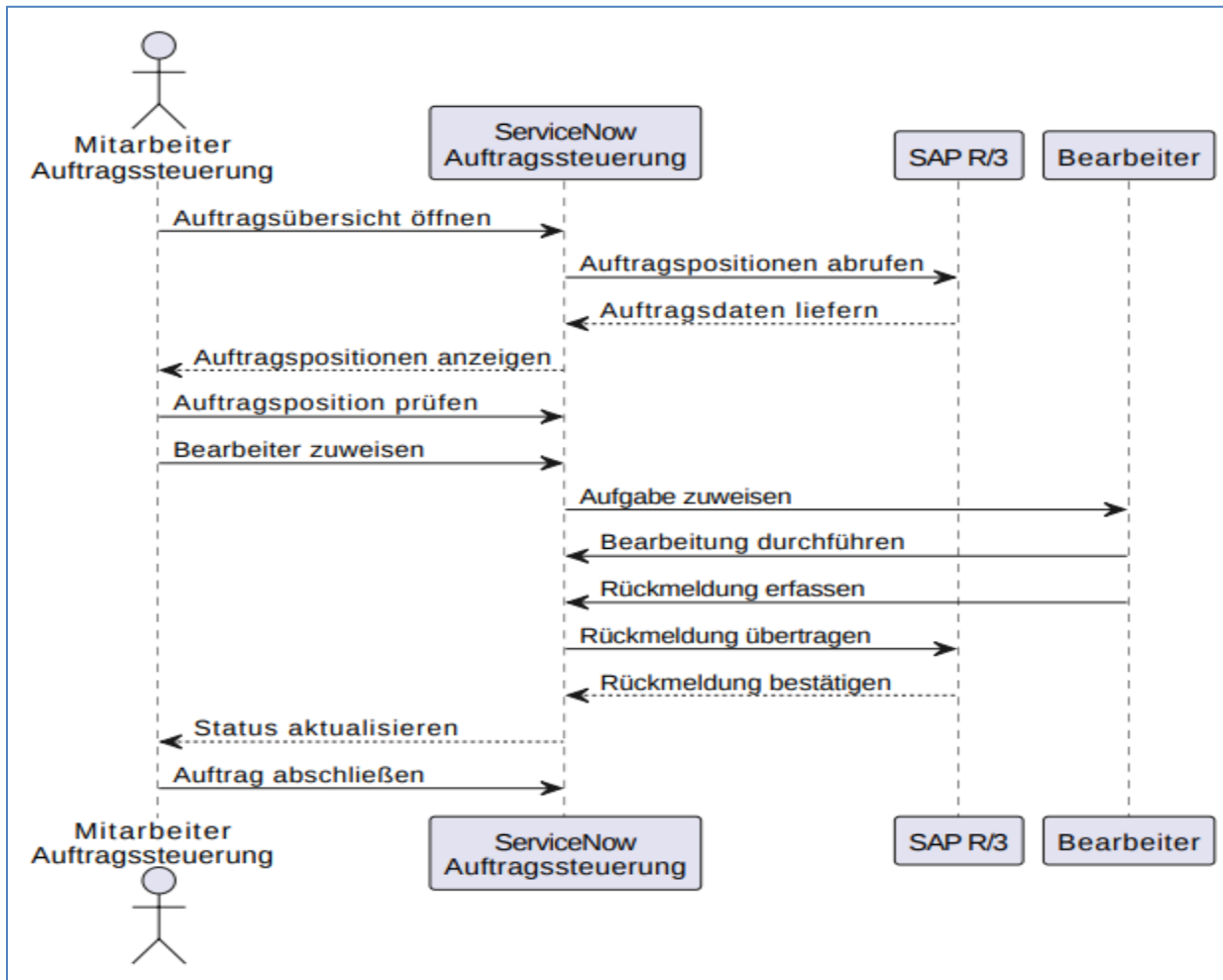
Aktivitätsdiagramm

Das Aktivitätsdiagramm beschreibt den Ablauf der Bearbeitung einer Auftragsposition von der Übernahme bis zur Archivierung. Dieser Diagrammtyp eignet sich besonders zur Darstellung von Geschäftsprozessen, Entscheidungen und Prozesslogik.



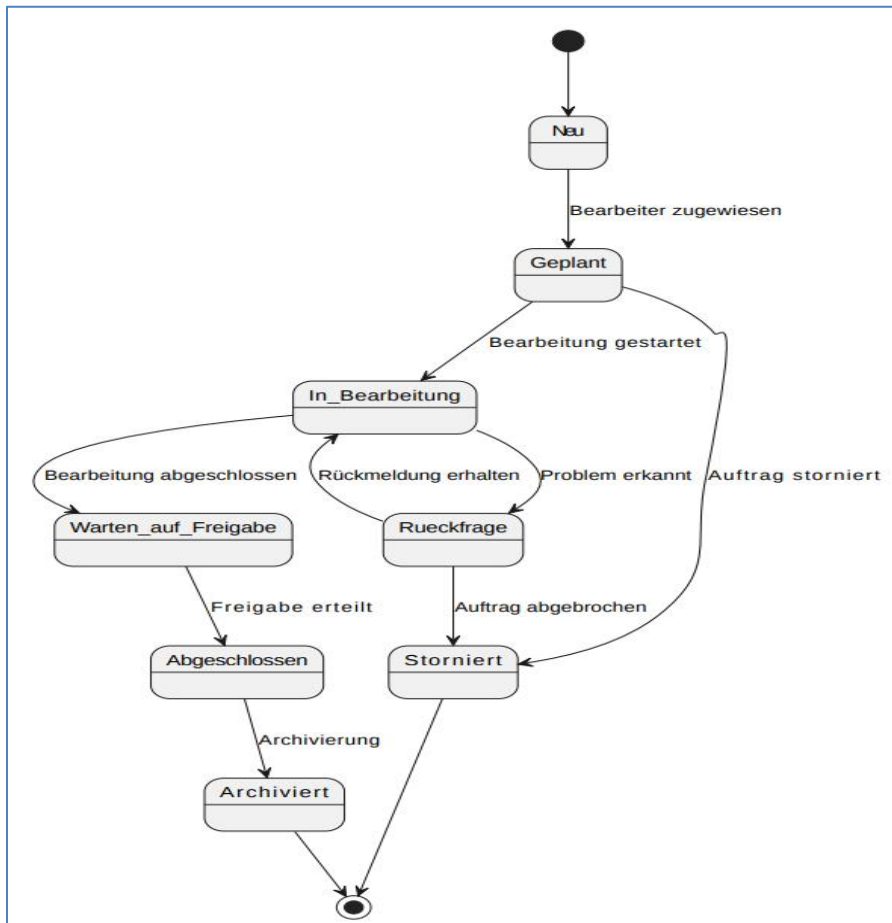
Sequenzdiagramm

Das Sequenzdiagramm zeigt die zeitliche Kommunikation zwischen SAP R/3, ServiceNow und den beteiligten Nutzern. Dieser Diagrammtyp eignet sich besonders zur Darstellung von Nachrichtenflüssen und Interaktionen zwischen Systemen und Akteuren.



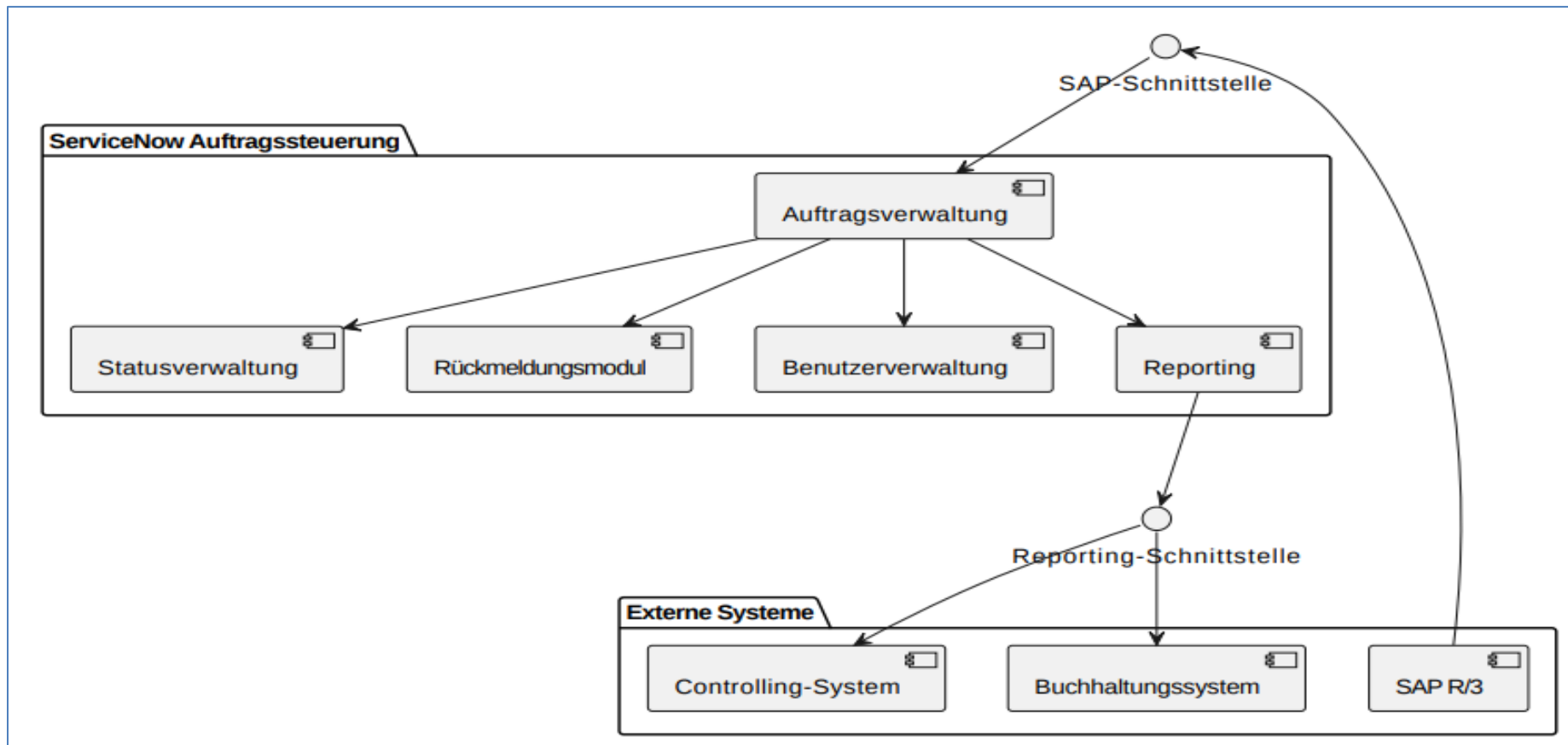
Zustandsdiagramm

Das Zustandsdiagramm zeigt die verschiedenen Zustände einer Auftragsposition während ihres Lebenszyklus. Dieser Diagrammtyp eignet sich besonders zur Modellierung von Statuswechseln und Zustandsübergängen innerhalb eines Prozesses.



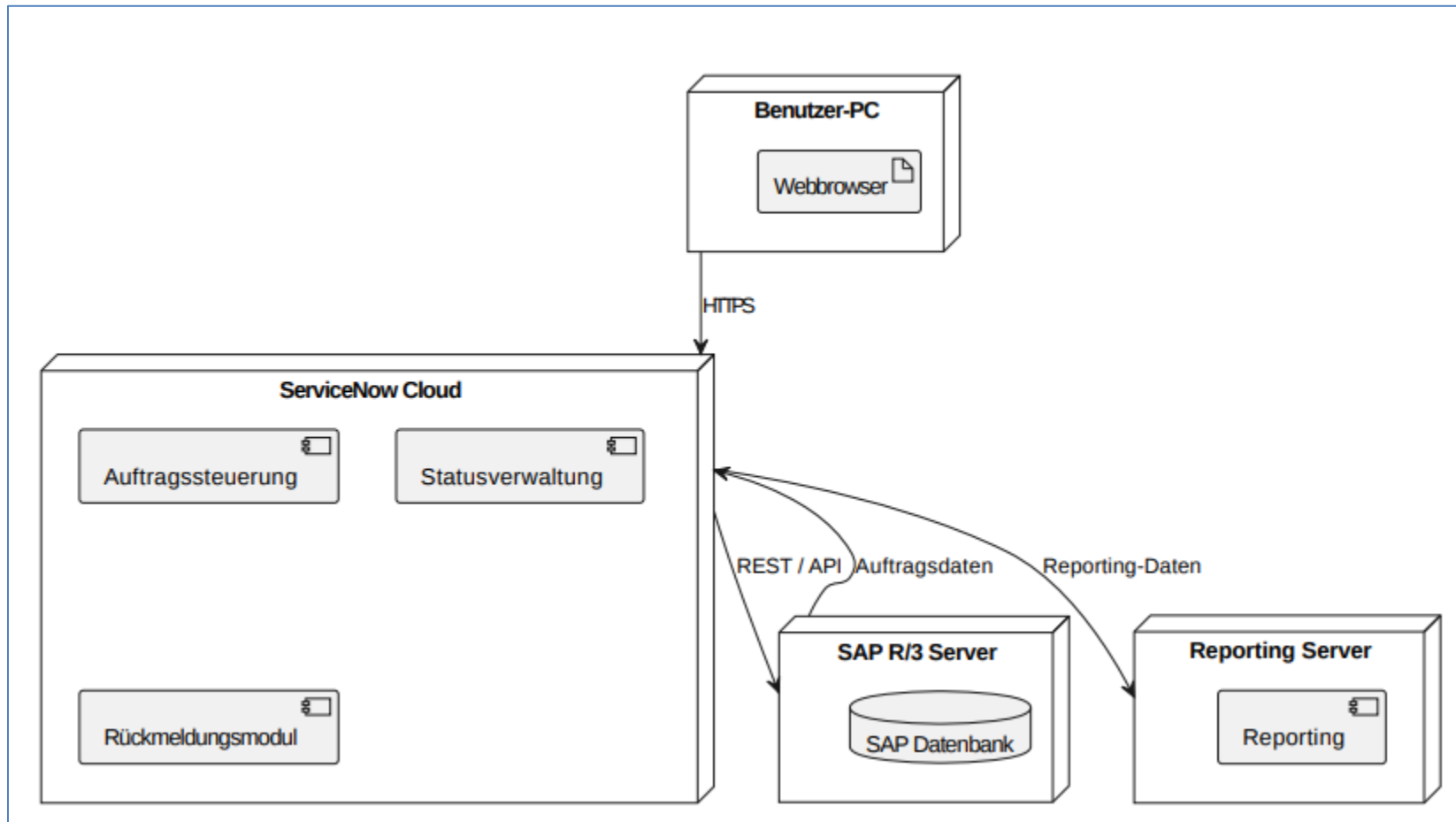
Komponentendiagramm

Das Komponentendiagramm zeigt die technischen Hauptkomponenten des Systems sowie deren Schnittstellen und Kommunikationsbeziehungen. Dieser Diagrammtyp eignet sich besonders zur Darstellung der Systemarchitektur und technischer Abhängigkeiten.



Deploymentdiagramm

Das Deploymentdiagramm zeigt die Verteilung der Systemkomponenten auf unterschiedliche technische Umgebungen und Systeme. Dieser Diagrammtyp eignet sich besonders zur Darstellung der Infrastruktur sowie der Kommunikation zwischen Clients, Servern und Plattformen.

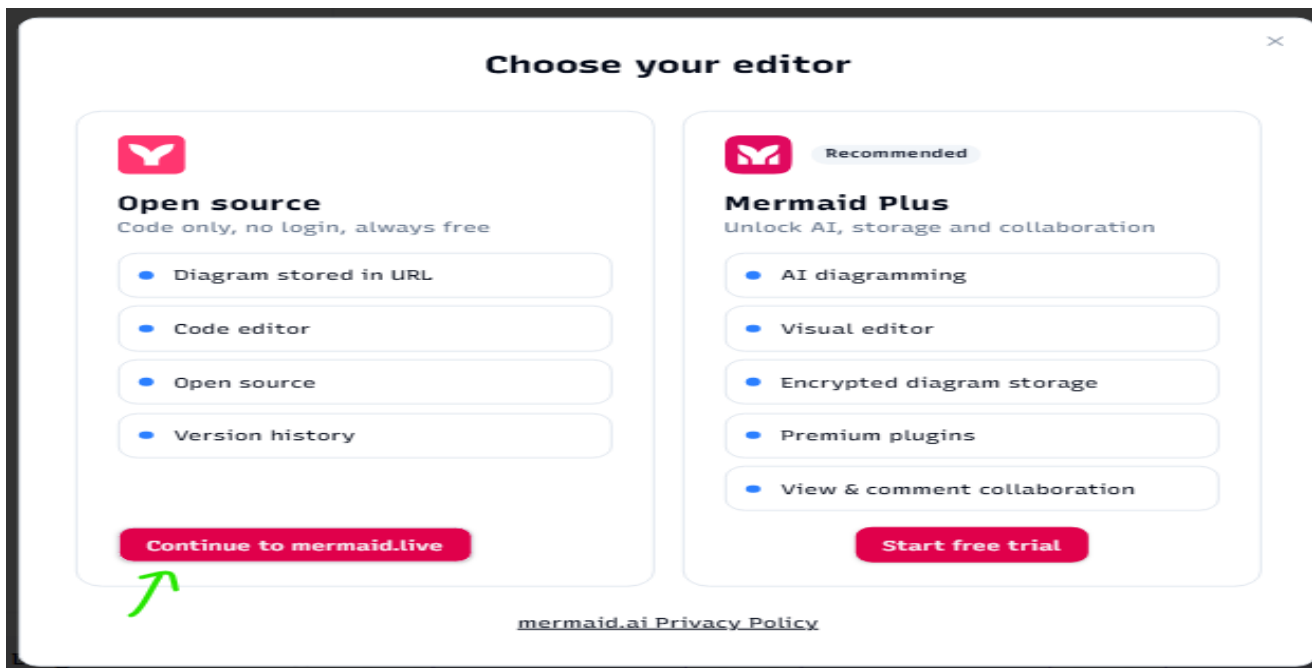


Teil B

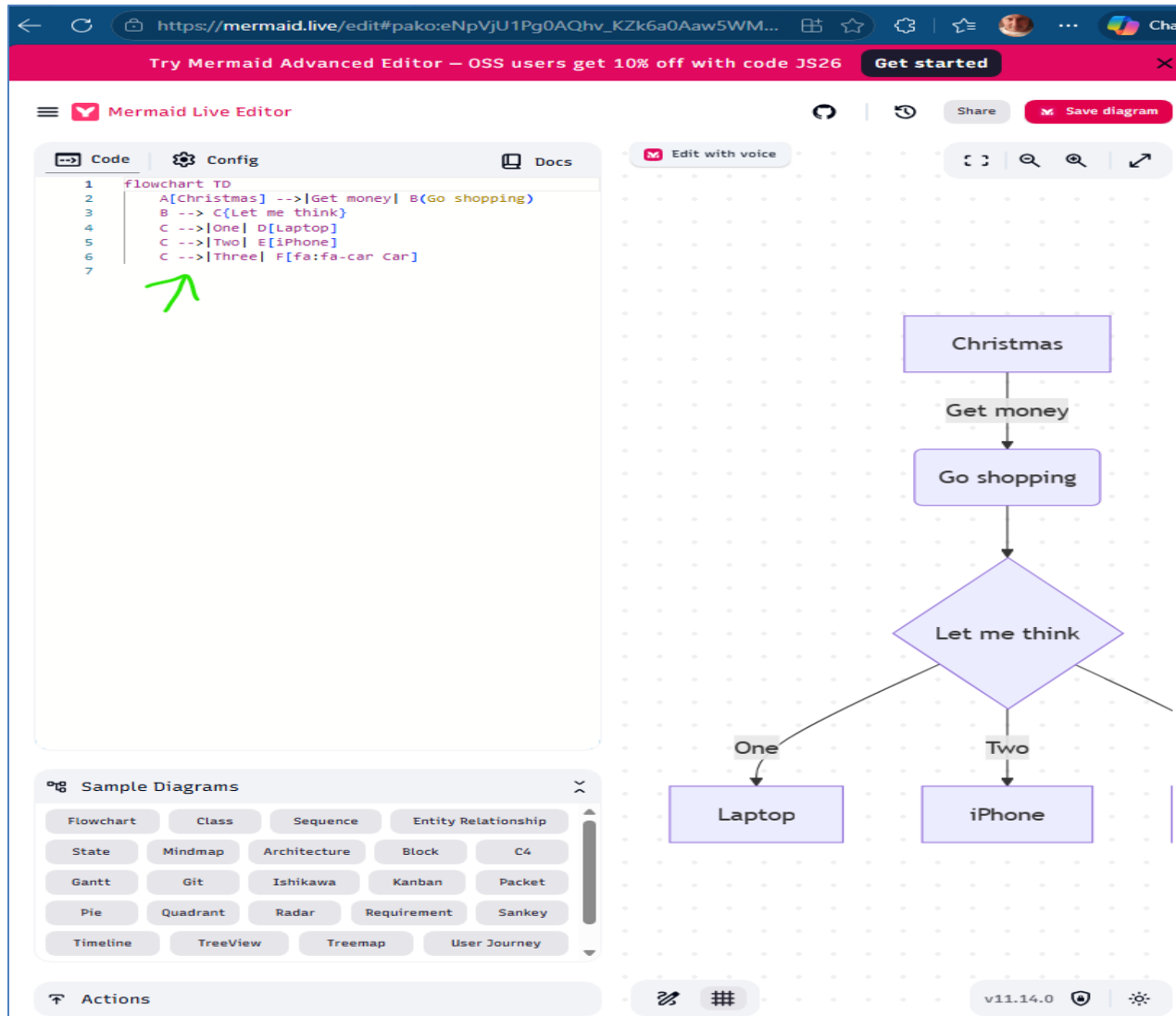
Für Teil B wurde Mermaid verwendet, da Diagramme dort textbasiert beschrieben und direkt durch GenAI erzeugt sowie im Live Editor angezeigt werden können. Über den Link bin ich dort hingekommen: <https://mermaid.live/>

Die Diagramme wurden auf Basis der eigenen Projektanalyse erstellt. GenAI wurde unterstützend zur Strukturierung einzelner Diagramminhalte und zur Erstellung eines Mermaid-Beispiels verwendet; die fachliche Prüfung und Anpassung erfolgte eigenständig.

Zuerst habe ich den Mermaid Life Editor geöffnet und hier entsprechend dabei ausgewählt:



Dann habe ich den alten Beispielcode links gelöscht.



Klassendiagramm wurde begonnen mit dem folgenden Code

Dann den Code dort eingefügt, welchen die KI generiert hat nach meinen Vorgaben:

```
classDiagram
```

```
class Auftrag {  
    +auftragsnummer  
    +kunde  
    +erstelldatum  
}
```

```
class Auftragsposition {  
    +positionsnummer  
    +beschreibung  
    +prioritaet  
}
```

```
class Benutzer {  
    +benutzerid  
    +name  
    +rolle  
}
```

```
class Bearbeiter {  
    +mitarbeiterNummer  
    +zuständigkeit  
}
```

```
class Fachbereich {  
    +bereichsName  
    +kuerzel
```

```
}
```

```
class Status {  
  +statusName  
  +farbwert  
}
```

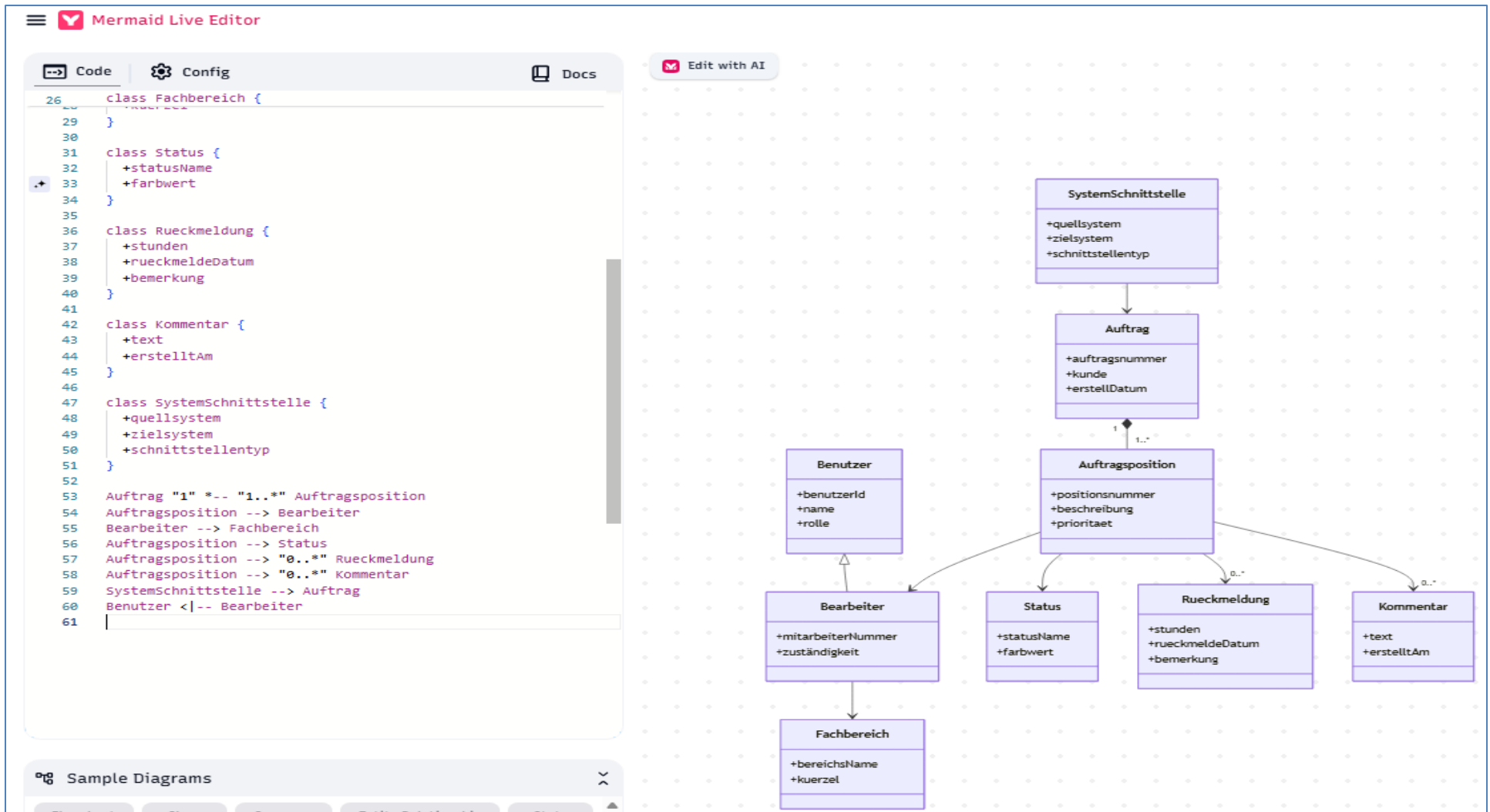
```
class Rueckmeldung {  
  +stunden  
  +rueckmeldeDatum  
  +bemerkung  
}
```

```
class Kommentar {  
  +text  
  +erstelltAm  
}
```

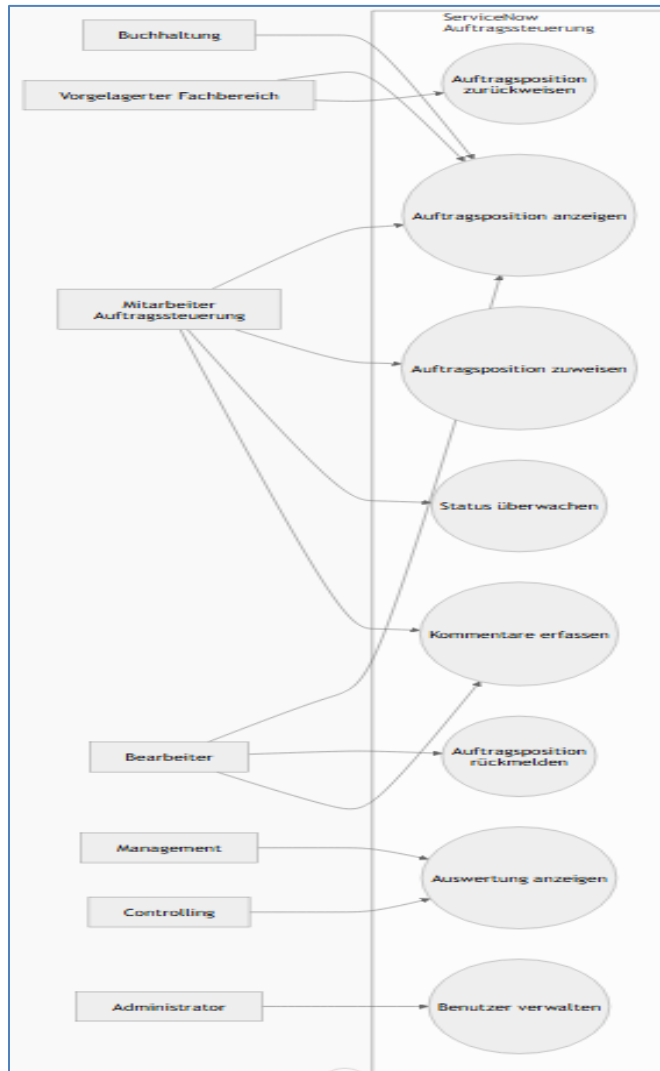
```
class SystemSchnittstelle {  
  +quellsystem  
  +zielsystem  
  +schnittstellentyp  
}
```

```
Auftrag "1" *-- "1..*" Auftragsposition  
Auftragsposition --> Bearbeiter  
Bearbeiter --> Fachbereich  
Auftragsposition --> Status  
Auftragsposition --> "0..*" Rueckmeldung  
Auftragsposition --> "0..*" Kommentar  
SystemSchnittstelle --> Auftrag  
Benutzer <|-- Bearbeiter
```

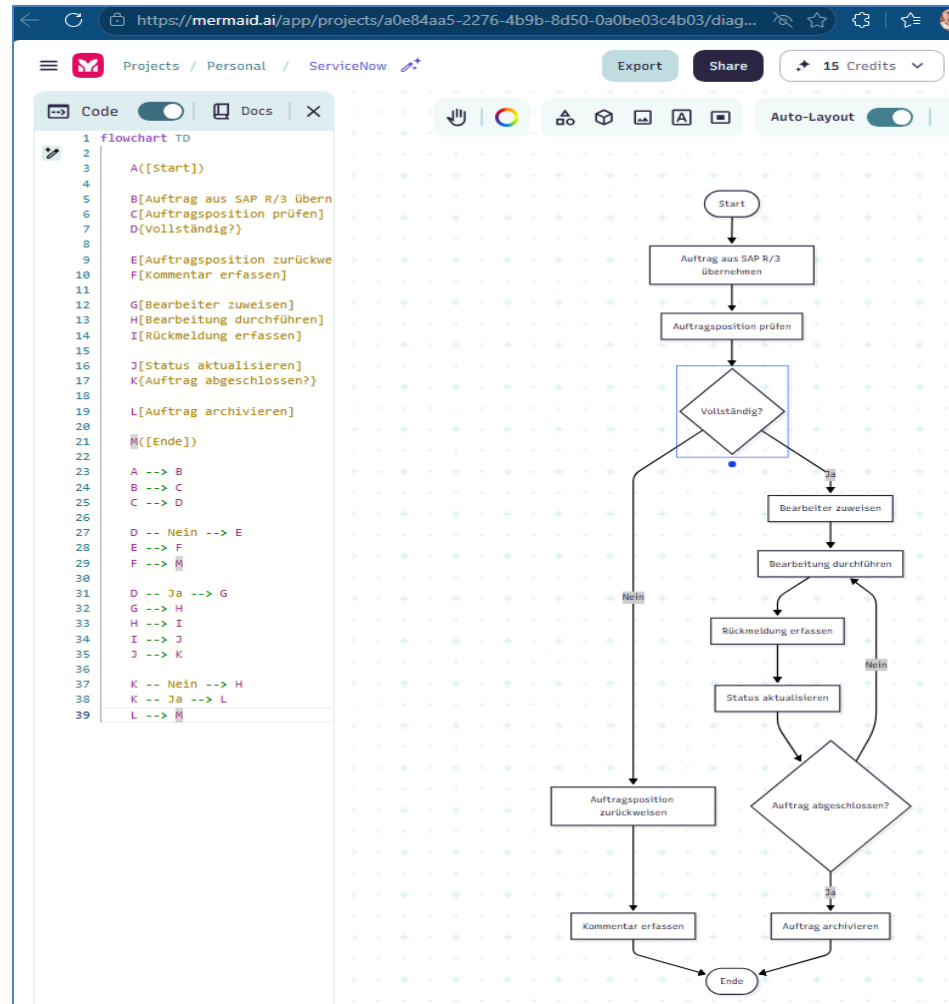
Auf der rechten Seite erschien dann das entsprechende **Klassendiagramm**:



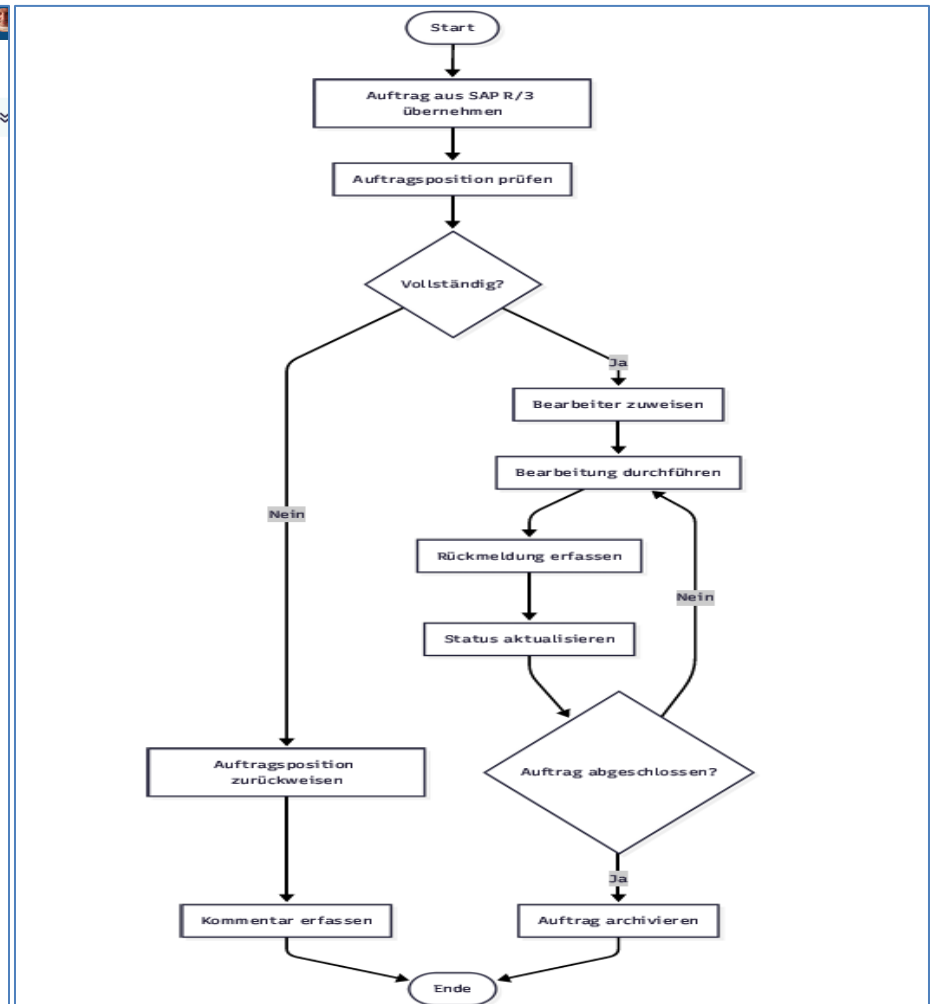
Anwendungsfalldiagramm:



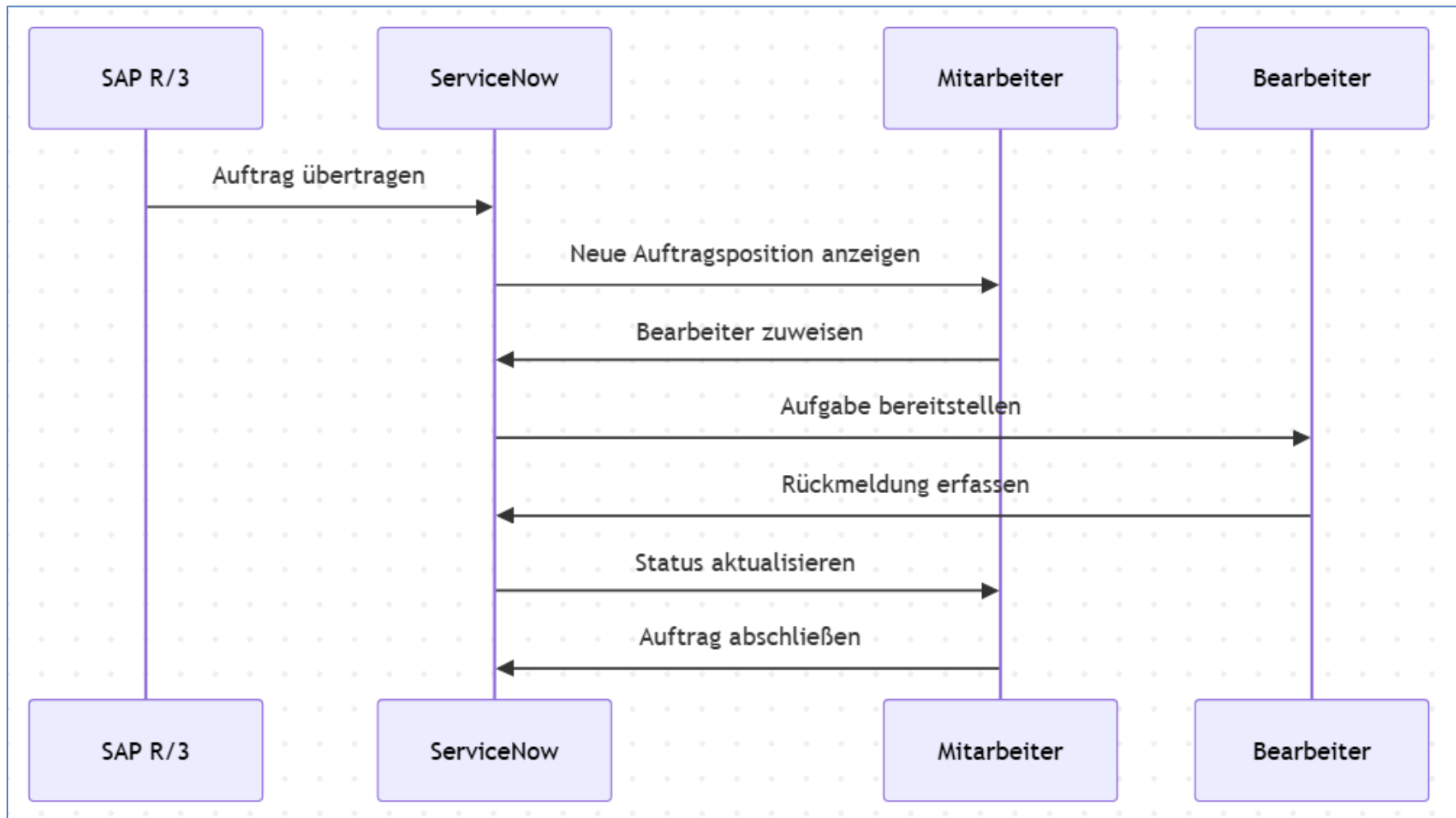
Aktivitätsdiagramm per Mermaid erstellt:



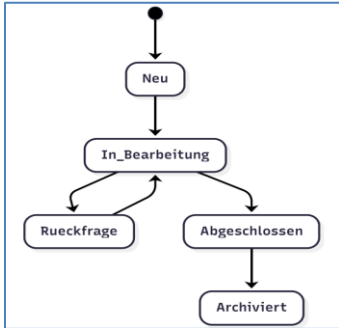
Hier nochmal als exportiertes und per Screenshot kopiertes Aktivitätsdiagramm zum Prozess SAP R/3 - ServiceNow:



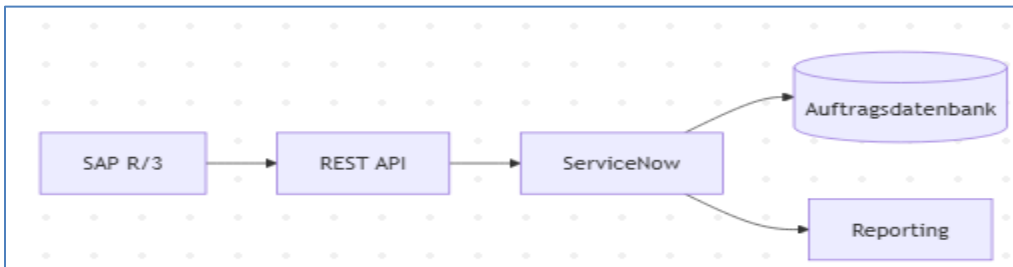
Sequenzdiagramm



Zustandsdiagramm



Komponentendiagramm



Deploymentdiagramm

